

UniSENAI Joinville
Curso: Análise e Desenvolvimento de Sistemas (ADS)
Aluna: Ana Paula dos Reis
Data: 07/07/2026

SEGURANÇA DE SOFTWARE

Princípios, ameaças e boas práticas para a proteção de sistemas

Joinville
2026

SUMÁRIO

- 1 INTRODUÇÃO
- 2 PRINCÍPIOS DE SEGURANÇA DA INFORMAÇÃO
 - 2.1 Confidencialidade
 - 2.2 Integridade
 - 2.3 Disponibilidade

2.4 Autenticidade e não repúdio

3 AMEAÇAS COMUNS NO DESENVOLVIMENTO DE SOFTWARE

3.1 Controle de acesso quebrado e configuração incorreta de segurança

3.2 Falhas na cadeia de suprimento de software

3.3 Falhas criptográficas e injeção de código

3.4 Falhas de projeto, autenticação e tratamento de exceções

4 BOAS PRÁTICAS DE SEGURANÇA NO CICLO DE DESENVOLVIMENTO

5 SEGURANÇA NO CÓDIGO-FONTE

6 AUTENTICAÇÃO E AUTORIZAÇÃO

7 TESTES DE SEGURANÇA

8 CONCLUSÃO

REFERÊNCIAS

1 INTRODUÇÃO

A segurança deixou de ser um item opcional no desenvolvimento de sistemas e passou a ocupar posição central entre os requisitos de qualidade de uma aplicação, ao lado de atributos como desempenho, usabilidade e manutenibilidade. Reznick e Monson-Haefel destacam que a engenharia de software profissional exige que decisões técnicas sejam tomadas com responsabilidade sobre seus efeitos para usuários e organizações, e a segurança se encaixa diretamente nesse compromisso, já que uma falha de proteção compromete tanto a confiança do usuário quanto a viabilidade do próprio produto (REZNICK; MONSON-HAEFEL, 2012).

O crescimento da dependência de sistemas informatizados conectados à internet, somado à quantidade crescente de dados sensíveis armazenados por empresas e órgãos públicos, tornou qualquer falha de segurança um evento potencialmente grave. Vazamentos de dados, invasões e indisponibilidade de serviços podem gerar prejuízos financeiros diretos, sanções regulatórias, perda de reputação e violação de direitos fundamentais dos usuários, como privacidade e proteção de dados pessoais. Nesse cenário, a segurança precisa ser tratada como uma propriedade do sistema construída ao longo de todo o ciclo de vida do software, e não como uma camada adicionada apenas na etapa final de implantação.

Este trabalho tem como objetivo pesquisar e apresentar os principais aspectos relacionados à segurança no desenvolvimento de software, contemplando os princípios fundamentais que orientam a proteção de sistemas, as ameaças mais relevantes documentadas pela comunidade de segurança da informação, as boas práticas de codificação segura, os mecanismos de autenticação e autorização e os testes empregados para identificar vulnerabilidades antes que sejam exploradas. Busca-se, com isso, demonstrar como a segurança deve permanecer presente em todas as fases do desenvolvimento, do levantamento de requisitos à manutenção em produção, protegendo simultaneamente os dados, os usuários e a organização responsável pelo sistema.

2 PRINCÍPIOS DE SEGURANÇA DA INFORMAÇÃO

Os princípios de segurança da informação aplicados ao desenvolvimento de software formam a base conceitual sobre a qual se apoiam todas as práticas de proteção de sistemas. Historicamente, esses princípios foram sintetizados na tríade CID, formada pela confidencialidade, pela integridade e pela disponibilidade, à qual se somam a autenticidade e o não repúdio como complementos indispensáveis em sistemas que lidam com identidade e transações.

2.1 Confidencialidade

A confidencialidade garante que a informação seja acessada apenas por pessoas ou processos devidamente autorizados. No contexto do desenvolvimento de software, esse princípio se traduz em mecanismos de controle de acesso, criptografia de dados sensíveis em trânsito e em repouso, e restrição de privilégios conforme a função de cada usuário dentro do sistema. Um sistema confidencial não impede necessariamente que dados sejam capturados por um invasor, mas garante que, mesmo capturados, esses dados permaneçam ininteligíveis e inúteis sem a devida autorização de acesso.

2.2 Integridade

A integridade assegura que os dados não sejam alterados de forma indevida, seja por ataques maliciosos, seja por falhas acidentais de hardware, software ou operação humana. Mecanismos como funções de hash criptográfico, assinaturas digitais e controle de

versionamento contribuem para que as informações permaneçam íntegras ao longo de todo o seu ciclo de vida, permitindo inclusive detectar alterações não autorizadas após o fato, o que é essencial em auditorias e investigações de incidentes.

2.3 Disponibilidade

A disponibilidade refere-se à garantia de que o sistema e seus dados estejam acessíveis sempre que forem necessários pelos usuários legítimos. Ataques de negação de serviço, distribuídos ou não, falhas de infraestrutura, erros de configuração e até mesmo picos inesperados de demanda são fatores que podem comprometer a disponibilidade de uma aplicação. Garantir esse princípio exige investimentos em redundância, balanceamento de carga, planos de recuperação de desastres e monitoramento contínuo da saúde do sistema.

2.4 Autenticidade e não repúdio

Além da tríade CID, outros dois princípios são relevantes para sistemas que envolvem identidade digital e transações. A autenticidade garante que a identidade de um usuário, dispositivo ou sistema é de fato verdadeira, evitando que um agente malicioso se passe por outra parte legítima. O não repúdio, por sua vez, impede que uma parte negue a autoria de uma ação realizada no sistema, sendo normalmente garantido por meio de registros de auditoria, assinaturas digitais e trilhas de log confiáveis. Esses princípios devem orientar todas as decisões de arquitetura, design e implementação do software desde as fases iniciais do projeto, característica reconhecida na literatura de gestão de projetos como parte da disciplina de levantamento e especificação de requisitos, que deve contemplar explicitamente requisitos não funcionais de segurança ao lado dos requisitos funcionais do sistema (O'REGAN, 2025).

3 AMEAÇAS COMUNS NO DESENVOLVIMENTO DE SOFTWARE

As ameaças à segurança de software evoluem constantemente, acompanhando o avanço das tecnologias e das técnicas empregadas por atacantes. A OWASP (Open Worldwide Application Security Project) mantém uma das referências mais reconhecidas mundialmente sobre o tema, o OWASP Top 10, documento que consolida as vulnerabilidades mais críticas encontradas em aplicações web a partir da análise de dados reais de segurança e de pesquisas realizadas com profissionais da área. A edição mais recente, o OWASP Top 10:2025, foi

anunciada em novembro de 2025 durante a conferência OWASP Global AppSec, em Washington, e teve sua versão final publicada em janeiro de 2026, com base na análise de mais de 175 mil registros de vulnerabilidades conhecidas (CVEs) e em pesquisas junto a milhares de organizações (PARASOFT, 2026). Essa edição trouxe duas categorias inteiramente novas e reorganizou a ordem de importância das ameaças já conhecidas, refletindo mudanças reais no comportamento de atacantes ao longo dos últimos anos (FASTLY, 2025).

3.1 Controle de acesso quebrado e configuração incorreta de segurança

O controle de acesso quebrado (Broken Access Control) permanece na primeira posição do OWASP Top 10:2025, mantendo-se como a vulnerabilidade mais crítica das últimas edições. Ocorre quando usuários conseguem acessar informações ou executar ações para as quais não possuem autorização, seja pela manipulação de parâmetros em URLs, seja pela ausência de validações consistentes de permissão em rotas e endpoints da aplicação. Na edição de 2025, essa categoria passou a incorporar explicitamente falhas de autorização em nível de objeto e de função em APIs, além de absorver a falsificação de solicitação do lado do servidor, que anteriormente era tratada como categoria isolada (FASTLY, 2025).

A configuração incorreta de segurança (Security Misconfiguration) apresentou a subida mais expressiva do ranking, saltando da quinta para a segunda posição. Esse avanço reflete um cenário em que equipes realizam implantações contínuas de software sem que a verificação de segurança acompanhe o mesmo ritmo, criando janelas de exposição sempre que uma configuração insegura, como credenciais padrão não alteradas, permissões públicas em serviços de nuvem ou recursos de depuração habilitados em produção, permanece ativa entre uma varredura de segurança e outra (GITLAB, 2026).

3.2 Falhas na cadeia de suprimento de software

As falhas na cadeia de suprimento de software (Software Supply Chain Failures) constituem uma das duas categorias inéditas da edição 2025, ocupando a terceira posição do ranking. Trata-se de uma expansão da antiga categoria de componentes desatualizados e vulneráveis, agora estendida para abranger todo o processo de construção, distribuição e atualização de software, incluindo bibliotecas de terceiros, pacotes de código aberto e os próprios pipelines de integração e entrega contínua. Ainda que apresente a menor incidência entre as

categorias testadas automaticamente, essa é a categoria com as maiores pontuações médias de explorabilidade e impacto entre todas as vulnerabilidades catalogadas, o que a torna particularmente perigosa (GITLAB, 2026). O caso do Log4Shell, vulnerabilidade identificada em uma biblioteca de registro de logs amplamente utilizada, ilustra bem esse risco: uma falha em um único componente comprometeu milhares de aplicações ao redor do mundo, evidenciando como a segurança de um sistema depende também da segurança de suas dependências.

O NIST (National Institute of Standards and Technology), órgão de padronização do governo dos Estados Unidos, publicou a Secure Software Development Framework (SSDF), documentada na publicação especial SP 800-218, justamente como resposta a incidentes dessa natureza, entre eles o ataque à SolarWinds. O framework organiza um conjunto de práticas em quatro grandes grupos, a preparação da organização, a proteção do software, a produção de software bem protegido e a resposta a vulnerabilidades, oferecendo um vocabulário comum para que fornecedores e consumidores de software discutam requisitos de segurança ao longo de toda a cadeia produtiva (NIST, 2022).

3.3 Falhas criptográficas e injeção de código

As falhas criptográficas (Cryptographic Failures) ocupam a quarta posição na edição 2025, descendo da segunda posição que ocupavam em 2021, o que reflete uma adoção mais consistente de TLS e de conjuntos de cifras mais robustos por parte da indústria (PARASOFT, 2026). Essa categoria envolve o uso de algoritmos obsoletos, a ausência de criptografia em dados sensíveis e a má gestão de chaves criptográficas, fatores que podem permitir a interceptação de credenciais e a exposição de dados confidenciais mesmo quando outros controles de segurança estão corretamente implementados.

A injeção de código, que inclui o SQL Injection e o Cross-Site Scripting (XSS), caiu para a quinta posição do ranking, mas continua sendo uma das vulnerabilidades mais perigosas e facilmente exploráveis, aparecendo em incidentes de alto impacto mais de duas décadas depois de ter sido documentada pela primeira vez pela OWASP. A injeção ocorre quando uma aplicação envia dados não confiáveis para um interpretador de comandos sem a devida validação ou tratamento. O SQL Injection permite que um atacante manipule consultas ao banco de dados, podendo ler, alterar ou excluir informações sem autorização, enquanto o XSS possibilita a

injeção de scripts maliciosos em páginas visualizadas por outros usuários, o que pode levar ao roubo de sessões e credenciais.

3.4 Falhas de projeto, autenticação e tratamento de exceções

O projeto inseguro (Insecure Design) representa uma categoria mais ampla de falhas, relacionadas a controles de segurança ausentes ou ineficazes já na fase de concepção do sistema, antes mesmo da implementação do código. As falhas de autenticação, renomeadas de forma consistente entre as edições, continuam sendo um dos principais facilitadores de acesso não autorizado, tomada de conta e vazamento de dados, e ocorrem quando um sistema não verifica ou protege adequadamente a identidade de seus usuários, permitindo que um atacante se passe por um usuário legítimo por meio de senhas fracas, ausência de autenticação multifator ou falhas na gestão de sessões.

A segunda categoria inédita da edição 2025, o tratamento inadequado de condições excepcionais (Mishandling of Exceptional Conditions), reúne falhas relacionadas ao tratamento incorreto de erros, situações lógicas inesperadas e mecanismos que falham de forma insegura, permitindo acesso indevido quando um componente do sistema encontra uma condição anômala. Um tratamento de exceções mal projetado pode expor informações sensíveis em mensagens de erro, como estruturas de banco de dados e caminhos internos do servidor, ou permitir que falhas em cascata resultem em negação de serviço (FASTLY, 2025). Por fim, as falhas de registro e alerta de segurança (Security Logging and Alerting Failures) permanecem na nona posição, reforçando que um sistema de logs sem mecanismos efetivos de alerta é de utilidade limitada para a identificação tempestiva de incidentes reais.

4 BOAS PRÁTICAS DE SEGURANÇA NO CICLO DE DESENVOLVIMENTO

A adoção de boas práticas ao longo de todo o ciclo de desenvolvimento reduz de forma significativa a superfície de ataque de uma aplicação. Essas práticas começam antes mesmo da escrita do primeiro código, na fase de levantamento de requisitos, quando os requisitos de segurança devem ser definidos com o mesmo rigor aplicado aos requisitos funcionais do sistema, e se estendem pela modelagem de ameaças na fase de design, etapa em que a equipe antecipa possíveis vetores de ataque antes que a arquitetura do sistema esteja consolidada.

Entre as práticas mais recomendadas, destacam-se a validação e a sanitização de todas as entradas de dados fornecidas pelo usuário, de modo a impedir que dados não confiáveis sejam interpretados como comandos; a aplicação do princípio do menor privilégio, concedendo a cada usuário ou processo apenas as permissões estritamente necessárias para sua função; o uso de consultas parametrizadas para evitar injeção de SQL; a criptografia de dados sensíveis em trânsito e em repouso com algoritmos atualizados, como AES e TLS 1.3; a gestão segura de dependências, com verificação constante de vulnerabilidades conhecidas em bibliotecas de terceiros; e a revisão de código associada ao uso de ferramentas de análise estática de segurança durante o próprio desenvolvimento.

Essas práticas devem ser incorporadas à cultura da equipe de desenvolvimento por meio de uma abordagem conhecida como *security by design*, segundo a qual a segurança é considerada desde a concepção do sistema, e não apenas corrigida após a identificação de problemas em produção. O SSDF do NIST formaliza essa lógica ao recomendar que práticas de segurança sejam integradas a qualquer modelo de ciclo de vida de desenvolvimento adotado pela organização, seja ele em cascata, iterativo ou baseado em metodologias ágeis combinadas com DevOps, justamente porque poucos modelos de ciclo de vida tratam a segurança de forma explícita e detalhada por padrão (NIST, 2022).

5 SEGURANÇA NO CÓDIGO-FONTE

Escrever código seguro exige disciplina técnica e conhecimento das vulnerabilidades mais comuns associadas à linguagem e ao ambiente de execução utilizados. Entre os cuidados fundamentais está a padronização do tratamento de erros, evitando que mensagens de exceção exponham detalhes internos da aplicação, como a estrutura do banco de dados ou os caminhos de arquivos do servidor, prática diretamente relacionada à categoria de *mishandling of exceptional conditions* discutida anteriormente. Reznick e Monson-Haefel reforçam, nesse sentido, que a combinação entre testes automatizados e revisão de código humana eleva substancialmente a taxa de detecção de erros em relação a qualquer uma dessas técnicas isoladamente, o que se aplica diretamente à identificação de falhas de segurança que passariam despercebidas em uma bateria de testes puramente funcional (REZNICK; MONSON-HAEFEL, 2012).

Outro cuidado importante é evitar a concatenação direta de strings para formar comandos SQL ou de sistema operacional, substituindo essa prática por consultas parametrizadas e por bibliotecas seguras de mapeamento objeto-relacional. Também é recomendável adotar listas de permissão em vez de listas de bloqueio para a validação de entradas, já que é mais seguro definir explicitamente o que é permitido do que tentar prever todas as formas possíveis de ataque, dado que novas variações de payloads maliciosos surgem continuamente.

Ferramentas de análise estática de segurança e de análise dinâmica auxiliam na identificação automatizada de padrões de código inseguro e de vulnerabilidades em tempo de execução, respectivamente, sendo cada vez mais incorporadas aos pipelines de integração contínua das equipes de desenvolvimento. Essa incorporação compõe o conceito de DevSecOps, no qual a segurança é tratada como responsabilidade compartilhada por toda a equipe, e não apenas por um setor especializado isolado do restante do processo produtivo. O próprio SSDF do NIST situa a análise estática e a análise dinâmica, incluindo técnicas de fuzzing, como práticas esperadas dentro do grupo de tarefas voltado à produção de software bem protegido (NIST, 2022).

6 AUTENTICAÇÃO E AUTORIZAÇÃO

Autenticação e autorização são mecanismos distintos, porém complementares, fundamentais para a segurança de qualquer sistema. A autenticação tem como objetivo verificar a identidade de um usuário, geralmente por meio de credenciais como senha, biometria ou tokens, enquanto a autorização define quais ações e recursos esse usuário, uma vez autenticado, está habilitado a acessar. Confundir essas duas etapas é um erro comum que costuma abrir brechas de segurança, já que um sistema pode autenticar corretamente um usuário e, ainda assim, falhar ao restringir o que esse usuário autenticado pode fazer.

Entre as práticas recomendadas para autenticação, destacam-se o uso de senhas com política de complexidade adequada, o armazenamento de senhas utilizando funções de hash seguras com salt, e a implementação de autenticação multifator, que adiciona uma camada extra de segurança ao exigir um segundo fator de verificação além da senha, como um código enviado ao dispositivo do usuário ou gerado por um aplicativo autenticador. A ausência de autenticação

multifator continua sendo apontada como um dos fatores que mais facilitam o comprometimento de contas, mesmo quando a senha original é razoavelmente robusta.

Para a autorização, é recomendada a adoção de modelos como o RBAC, sigla para controle de acesso baseado em papéis, que atribui permissões conforme funções previamente definidas dentro do sistema, facilitando tanto a manutenção quanto a auditoria dos acessos concedidos. É essencial, ainda, validar as permissões em todas as camadas da aplicação, especialmente no backend, já que validações realizadas apenas na interface do usuário podem ser facilmente contornadas por um atacante que interaja diretamente com a API do sistema. A importância desses mecanismos reside no fato de que grande parte dos incidentes de segurança está relacionada a falhas de identificação e autenticação, como o uso de credenciais padrão não alteradas, senhas fracas ou ausência de expiração de sessões, o que reforça a necessidade de atenção redobrada a esse aspecto do sistema.

7 TESTES DE SEGURANÇA

Os testes de segurança têm como finalidade identificar vulnerabilidades no sistema antes que sejam exploradas por agentes maliciosos, complementando os testes funcionais tradicionais, que raramente são suficientes para revelar falhas de segurança. Entre os principais tipos de testes utilizados estão o teste de penetração, que consiste na simulação controlada de ataques reais realizada por profissionais especializados com o objetivo de identificar e explorar vulnerabilidades presentes no sistema, e a análise estática de segurança, que examina o código-fonte da aplicação sem executá-la, em busca de padrões de codificação inseguros.

A análise dinâmica de segurança complementa a análise estática ao testar a aplicação em execução, simulando ataques externos para identificar falhas expostas em tempo real, enquanto a análise de composição de software verifica as bibliotecas e dependências de terceiros utilizadas pela aplicação, identificando vulnerabilidades já conhecidas e catalogadas publicamente, prática que ganhou ainda mais relevância diante do crescimento das falhas na cadeia de suprimento discutidas anteriormente. Por fim, os testes de auditoria de logs e monitoramento verificam a capacidade do sistema de registrar e alertar sobre eventos de segurança relevantes em tempo hábil, evitando que a categoria de falhas de registro e alerta se materialize em incidentes não detectados.

A combinação dessas diferentes abordagens de teste permite uma cobertura mais abrangente das possíveis vulnerabilidades do sistema, sendo recomendável que os testes de segurança sejam realizados de forma contínua, integrados ao pipeline de desenvolvimento, e não apenas como uma etapa isolada ao final do projeto. Essa integração contínua está alinhada ao próprio conceito de qualidade de software defendido por Reznick e Monson-Haefel, para quem a qualidade não é uma etapa de verificação posterior, mas uma propriedade construída de forma incremental ao longo de todo o processo de desenvolvimento (REZNICK; MONSON-HAEFEL, 2012).

8 CONCLUSÃO

A pesquisa realizada evidencia que a segurança de software é um requisito transversal, que deve permanecer presente em todas as etapas do ciclo de desenvolvimento, desde o levantamento de requisitos até a manutenção do sistema em produção. Os princípios de confidencialidade, integridade e disponibilidade, complementados pela autenticidade e pelo não repúdio, orientam as decisões de projeto, enquanto o conhecimento das ameaças mais relevantes, consolidadas em referências como o OWASP Top 10:2025 e sistematizadas em frameworks como o SSDF do NIST, permite que equipes de desenvolvimento antecipem e mitiguem riscos antes que se tornem incidentes reais.

A atualização trazida pela edição 2025 do OWASP Top 10, com a inclusão das falhas na cadeia de suprimento de software e do tratamento inadequado de condições excepcionais, demonstra que as ameaças acompanham a evolução da própria forma como o software é construído, distribuído e mantido, cada vez mais dependente de componentes de terceiros e de pipelines automatizados de entrega contínua. Isso reforça que a segurança não pode se limitar ao código escrito internamente pela equipe, precisando alcançar toda a cadeia de produção do software.

Conclui-se que a adoção de boas práticas de codificação segura, aliada a mecanismos robustos de autenticação e autorização, bem como à realização sistemática de testes de segurança integrados ao pipeline de desenvolvimento, constitui a estratégia mais eficaz para a construção de sistemas confiáveis. Como reforça o princípio do security by design, a segurança não deve ser tratada como uma etapa final ou opcional do desenvolvimento, mas como um valor incorporado

à cultura de toda a equipe responsável pelo software, protegendo usuários, dados e a própria organização contra ataques e falhas.

REFERÊNCIAS

FASTLY. The New 2025 OWASP Top 10 List: What Changed, and What You Need to Know. Disponível em: <https://www.fastly.com/blog/new-2025-owasp-top-10-list-what-changed-what-you-need-to-know>. Acesso em: 07 jul. 2026.

GITLAB. OWASP Top 10 2025: what's changed and why it matters. Disponível em: <https://about.gitlab.com/blog/2025-owasp-top-10-whats-changed-and-why-it-matters/>. Acesso em: 07 jul. 2026.

NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY (NIST). Secure Software Development Framework (SSDF) Version 1.1: recommendations for mitigating the risk of software vulnerabilities. SP 800-218. Gaithersburg, MD: NIST, 2022. Disponível em: <https://doi.org/10.6028/NIST.SP.800-218>. Acesso em: 07 jul. 2026.

O'REGAN, Gerard. Guide to Software Project Management. 2. ed. Cham: Springer Nature Switzerland, 2025. (Undergraduate Topics in Computer Science).

OWASP FOUNDATION. OWASP Top 10:2025. Disponível em: <https://owasp.org/Top10/2025/>. Acesso em: 07 jul. 2026.

PARASOFT. OWASP Top 10 2025: what changed & new vulnerabilities. Disponível em: <https://www.parasoft.com/blog/owasp-top-10/>. Acesso em: 07 jul. 2026.

REZNICK, Alan; MONSON-HAEFEL, Richard. Software Development and Professional Practice. New York: Apress, 2012.